

AWR2944 FMCW/TWTT MATLAB Simulation Blueprint

A staged implementation plan from analytic truth model to two-node nonideal
digital twin

Summer 2026 - Saeed FMCW/TWTT Project

Curated research synthesis and engineering reference

Prepared 10 August 2026

This document is a project research aid, not a claim of novelty. It distinguishes source-supported prior art, engineering inference, and proposed simulation steps. Publisher/IEEE PDFs supplied by the user are retained in the archive for private research and are not represented as redistributable.

Contents

1	Design objective	4
2	Why complex baseband is the correct starting representation	4
3	Repository/code layout	4
4	Configuration model	5
5	V0: single-link analytic truth model	6
5.1	Inputs	6
5.2	Waveform	6
5.3	Dechirp	6
5.4	Estimators	6
5.5	V0 acceptance criteria	6
6	V0 plotting package	6
6.1	Figure A: chirp geometry	6
6.2	Figure B: dechirped signal	6
6.3	Figure C: delay sweep	7
7	V1: ideal reciprocal two-way FMCW timing	7
7.1	Recommended demonstration numbers	7
7.2	V1 acceptance criteria	7
8	V1 program-manager figure	7
9	V1.5: estimator-resolution demonstration	7
10	V2: independent time and frequency offsets	8
11	V2.1: clock skew and sample-rate offset	8
12	V2.2: chirp slope mismatch	8
13	V3: white-noise Monte Carlo	8
14	V4: phase noise	8
15	V5: systematic chirp distortion	9

16 V6: front-end and ADC fidelity	9
17 V7: multipath	9
18 V8: AWR2944-faithful configuration	9
19 Important hardware question: very low beat frequencies	10
20 V9: hardware-delay calibration model	10
21 V10: phase-coded node identity	10
22 Test philosophy	11
23 Numerical precision	11
24 Data products	11
25 Recommended first coding sprint	11
26 Definition of done for the first Saeed deliverable	11
27 What comes immediately after the first plots	12
28 Final validation against hardware	12
29 Reference anchors	12
30 Module contracts	13
31 Detailed unit-test matrix	13
32 First demo pseudocode	14
33 Two-way demo pseudocode	15
34 Estimator benchmark protocol	15
35 Monte Carlo experiment matrix	16
36 AWR2944 config extraction checklist	16
37 DCA1000/raw-data considerations	16
38 Low-frequency IF feasibility checklist	17

39 Two-board hardware characterization plan	17
40 Result-file schema	18
41 Plot style for Saeed/program manager	18
42 Software-quality gates	18
43 Milestone schedule from here	19
44 Stop conditions and debugging order	19

1 Design objective

Build one MATLAB codebase that can serve three roles without rewriting the physics: (1) a transparent ideal FMCW demonstration for Saeed/program-manager plots; (2) an ideal reciprocal two-way timing simulator that recovers propagation delay and clock offset; and (3) a progressively more realistic AWR2944-oriented precision simulator with clock, PLL, front-end, sampling, and coding impairments.

The design must prioritize verifiability over feature count. Every block needs a closed-form limiting case or a unit test. A block that cannot be independently tested should not be added to V0/V1.

2 Why complex baseband is the correct starting representation

A 77-GHz carrier does not need to be explicitly sampled to reproduce FMCW dechirping physics. Represent the chirp by its analytic phase, preserve exact fractional delays, and evaluate the mixer difference phase. This captures the information that reaches the IF/ADC while keeping the simulation sample rate tied to IF bandwidth rather than RF carrier frequency.

This approach also matches the philosophy of the Scheibelhofer MATLAB simulator: simulate on a time grid sufficient for IF/noise dynamics, then decimate to the final ADC rate. The first version can simplify further by generating the dechirped analytic signal exactly; later versions can insert oversampled IF filters and ADC decimation.

3 Repository/code layout

Recommended MATLAB layout:

```
10_SIMULATION_WORKSPACE/  
  README.md  
  config/  
    cfg_ideal_demo.m  
    cfg_awr_smoke_v1.m  
  src/  
    waveform/  
      fmcw_phase.m  
      fmcw_chirp.m  
      apply_fractional_delay.m  
    clocks/  
      local_time_map.m  
      clock_offset_model.m  
    channel/  
      free_space_delay.m  
      multipath_channel.m  
    rf/  
      dechirp.m  
      if_filter.m  
      quantize_adc.m  
    impairments/  
      add_cfo.m  
      add_slope_error.m
```

```
    add_phase_noise_psd.m
    add_chirp_nonlinearity.m
    add_group_delay.m
estimation/
    estimate_fft_peak.m
    estimate_phase_slope.m
    estimate_czt.m
    solve_twtt.m
plots/
    plot_chirp_geometry.m
    plot_beat_spectrum.m
    plot_twtt_summary.m
tests/
    test_ideal_delay.m
    test_fractional_delay.m
    test_twtt_recovery.m
    test_cfo_up_down.m
    test_slope_mismatch.m
demos/
    demo01_single_link_ideal.m
    demo02_twtt_ideal.m
    demo03_estimator_resolution.m
    demo04_awr_parameters.m
```

4 Configuration model

Use one explicit struct. Avoid hidden globals and toolbox-default parameters.

```
cfg.c = 299792458;
cfg.f0 = 77e9;
cfg.B = ...;
cfg.Tc = ...;
cfg.S = cfg.B/cfg.Tc;
cfg.Fs = 10e6;
cfg.N = 256;

cfg.link.tau = 5e-9;
cfg.clock.theta = 100e-12;
cfg.clock.skew = 0;
cfg.clock.cfo = 0;

cfg.impair.awgn_snr_db = Inf;
cfg.impair.phase_noise_enable = false;
cfg.impair.nonlinearity_enable = false;
cfg.impair.group_delay_enable = false;
```

Convert every TI unit (GHz, MHz/us, ksps, us) at the configuration boundary. Internal calculations should remain SI.

5 V0: single-link analytic truth model

5.1 Inputs

f_0 , S , observation interval, one-way delay τ , complex amplitude α , optional initial phase.

5.2 Waveform

Generate phase

$$\phi(t) = \phi_0 + 2\pi f_0 t + \pi S t^2$$

and chirp $s(t) = e^{j\phi(t)}$. Apply the delay by evaluating $\phi(t - \tau)$, not by shifting the sample array. This keeps 10-ps-class test delays exact even when ADC sample spacing is tens or hundreds of nanoseconds.

5.3 Dechirp

With the selected convention,

$$z(t) = s(t)r^*(t).$$

The no-noise phase should be linear. Its slope must equal $2\pi S\tau$ to numerical precision.

5.4 Estimators

Implement two estimators immediately:

1. FFT peak for intuitive plotting and coarse sanity checks.
2. phase-slope least-squares estimator for transparent sub-bin frequency recovery.

The FFT is not allowed to be the ground-truth estimator in tests.

5.5 V0 acceptance criteria

- Relative error in recovered f_b below a numerical tolerance for noise-free cases.
- Fractional delays smaller than one ADC sample recovered correctly.
- Doubling S doubles the estimated beat.
- Switching mixer conjugation changes sign only.

6 V0 plotting package

6.1 Figure A: chirp geometry

Plot instantaneous frequency versus time for local TX and delayed RX. Annotate horizontal frequency separation $S\tau$. The plot should visually explain why a time shift becomes a frequency offset on a linear slope.

6.2 Figure B: dechirped signal

Show a short time trace of the complex beat phase or real IF plus a spectrum. Annotate theoretical and estimated f_b .

6.3 Figure C: delay sweep

Sweep τ over a reasonable range and plot estimated f_b versus τ . Overlay the theoretical line $S\tau$. This is an extremely strong program-manager plot because it shows linearity and validates the model over more than one hand-picked point.

7 V1: ideal reciprocal two-way FMCW timing

Create station A and B with equal rate/slope and a relative epoch offset θ . Do not yet simulate a separate physical PLL. Instead define the effective relative delays:

$$\delta_{AB} = \tau + \theta, \quad \delta_{BA} = \tau - \theta.$$

Generate two dechirped observations using the same waveform functions. Estimate signed f_{AB} and f_{BA} , then solve

$$\hat{\tau} = \frac{f_{AB} + f_{BA}}{2S}, \quad \hat{\theta} = \frac{f_{AB} - f_{BA}}{2S}.$$

7.1 Recommended demonstration numbers

Use the AWR-like slope $S = 29.982 \text{ MHz}/\mu\text{s}$ with $\tau = 5 \text{ ns}$ and $\theta = 100 \text{ ps}$. The resulting beats are separated by only about 6 kHz, making the point that sub-bin estimation matters without requiring absurdly tiny frequencies.

7.2 V1 acceptance criteria

- $\hat{\tau}$ and $\hat{\theta}$ recover injected values to noise-free numerical tolerance.
- Sign conventions are documented in code comments and unit tests.
- If $\theta = 0$, two beats are equal.
- If $\tau = 0$, beats are equal-and-opposite under the ideal equation.

8 V1 program-manager figure

Use either one composite figure or two clean figures:

- top: A-to-B and B-to-A beat spectra/time-frequency markers;
- bottom/right: a small equation box reporting injected and recovered TOF and clock offset.

Avoid showing dozens of radar DSP panels. The point is the transformation “two tiny time quantities $\rightarrow$ two easy beat frequencies $\rightarrow$ sum/difference recovery.”

9 V1.5: estimator-resolution demonstration

Before adding hardware noise, demonstrate why nearest-bin FFT is insufficient. For the current short ADC window, show:

1. raw FFT with the true beat between bins;
2. nearest-bin estimate;
3. zero-padded/interpolated display;

4. phase-slope estimate;
5. later CZT estimate.

Plot timing error in ps for each. This turns the “10 ps vs 39-kHz FFT bin” concern into a concrete signal-processing story.

10 V2: independent time and frequency offsets

Add a carrier-frequency offset Δf between the local and received chirps. For an up-chirp,

$$f_+ = \Delta f + S\tau_{eff}.$$

For a down-chirp,

$$f_- = \Delta f - S\tau_{eff}.$$

Use both to recover Δf and τ_{eff} . This reproduces the conceptual method in the 2007 cooperative synchronization paper and gives the model its first real “clock-rate/frequency” dimension.

11 V2.1: clock skew and sample-rate offset

Represent relative epoch as $\theta(t) = \theta_0 + \epsilon t$. Separately allow ADC sample-rate error $F_{s,B} = F_s(1 + \epsilon_s)$. Verify their signatures differ. The first creates physical/time-map evolution; the second distorts the digital time axis. In real hardware they may be related to the same reference, but the model should not assume that until the architecture is verified.

12 V2.2: chirp slope mismatch

Set $S_B = S_A + \Delta S$. The dechirped signal should become chirped rather than a pure tone. Unit test this by estimating instantaneous phase slope and verifying the predicted linear drift. This is important because a two-node estimator that assumes a stationary beat can silently report biased timing when slopes differ.

13 V3: white-noise Monte Carlo

Add post-dechirp complex AWGN first. Sweep SNR and run at least 10^3 Monte Carlo trials per operating point for initial plots, increasing as needed for tails. Record bias, standard deviation, RMS error, estimator failure rate, and computational time.

Suggested plots:

- $\sigma_{\hat{\tau}}$ versus SNR;
- $\sigma_{\hat{\theta}}$ versus SNR;
- estimator comparison: FFT/interpolated FFT/phase-slope/CZT;
- precision versus observation duration/chirp count;
- precision versus slope S at fixed SNR.

14 V4: phase noise

Do not begin with arbitrary phase jitter samples. Implement two modes:

Simple validation mode.

White frequency/phase process with a known analytic behavior, used only to validate plumbing.

PSD mode.

Generate colored phase noise whose one-sided/two-sided PSD convention is explicit. Use measured or documented masks later.

For monostatic/homodyne validation, use a common phase-noise realization for TX and LO so that the range-correlation difference $\phi(t) - \phi(t - \tau)$ emerges. For two independent stations, use separate realizations. For CFDDS-inspired experiments, process the two reciprocal observations jointly and measure how much of the noise-like term cancels.

15 V5: systematic chirp distortion

Implement deterministic $e_f(t)$ or $\phi_{sys}(t)$ functions:

1. quadratic curvature;
2. cubic curvature;
3. sinusoidal spur/ripple;
4. later, an imported measured residual trace.

For each, show beat spectrogram/phase residual and timing bias. Validate that setting the coefficients to zero reproduces V3 exactly.

16 V6: front-end and ADC fidelity

Following the Scheiblhofer block philosophy, introduce:

- TX/RX complex gain;
- leakage/direct coupling;
- IF low-pass/band-pass transfer function;
- per-path group delay;
- ADC clipping/full scale;
- quantization/ENOB;
- oversampled simulation clock followed by decimation to AWR/DCA sample rate.

Keep this layer separate from oscillator impairments so each contribution to timing error can be toggled independently.

17 V7: multipath

Use a direct active cross-link plus one or more delayed weaker paths. Sweep relative amplitude and excess delay. Evaluate bias of frequency-only, phase-only, and combined estimators. This is a likely source of significant real-world error in parking-lot/indoor tests.

18 V8: AWR2944-faithful configuration

Create a config parser or manual mapping from the actual profile used by the radar package. At minimum capture:

- start frequency;
- slope;
- ADC start time;
- sample rate and number of samples;
- ramp end time/idle time;
- chirps per frame and frame period;
- TX/RX selection;
- any known analog HPF/IF constraints relevant to low beat frequencies.

Then run the same V0/V1 tests with AWR-derived parameters. The goal is not to emulate every BSS register. It is to guarantee that the simulated waveform and acquisition window correspond to what the package actually configures.

19 Important hardware question: very low beat frequencies

Picosecond offsets can map to hundreds of hertz at the current slope. AWR analog HPF corners, leakage cancellation, DC offsets, and the need to avoid near-DC beat content can therefore matter. The early cooperative paper intentionally uses known frequency offsets to keep mixed signals away from DC. Before finalizing the hardware waveform, inspect AWR HPF/IF configuration and consider adding a deliberate beat-frequency offset that is algebraically removed in processing.

20 V9: hardware-delay calibration model

Give each station separate g^{TX} and g^{RX} delays plus frequency-dependent group delay. Run reciprocal solves before and after calibration. Distinguish:

- repeatability/precision of an unchanging setup;
- absolute time offset accuracy after fixed-delay calibration;
- drift with temperature, power cycle, RF channel, and configuration.

This distinction will likely become central if the goal is “10 ps.”

21 V10: phase-coded node identity

Only after V1–V3 are stable, add phase coding:

$$s_i(t) = s_{FMCW}(t)e^{j\phi_{c,i}(t)}.$$

Start with two orthogonal or low-cross-correlation code sequences. Demonstrate that a receiver can identify A versus B at different amplitudes. Then introduce propagation delay that misaligns code chips. Compare naive decoding with an alignment/group-delay strategy inspired by Uysal/Lampel/Kumbul.

Metrics:

- code detection probability;
- cross-correlation sidelobe/interference rejection;
- timing bias after coding;
- required ADC/IF bandwidth;
- SNR loss relative to uncoded FMCW;
- number of simultaneously identifiable nodes.

22 Test philosophy

Each physical mechanism gets three tests:

1. **zero test**: disabled mechanism changes nothing;
2. **analytic test**: one simple case matches a closed-form prediction;
3. **stress test**: large value produces the expected qualitative failure signature.

Do not accept a Monte Carlo plot as evidence that a block is correct unless the zero and analytic tests pass first.

23 Numerical precision

Use double precision throughout V0–V5. Avoid subtracting two enormous 77-GHz phase numbers when a mathematically equivalent baseband expression can be evaluated directly; catastrophic phase wrapping can obscure small timing differences. Where absolute RF phase is needed, reduce arguments modulo 2π carefully and compare against symbolic/analytic expressions.

24 Data products

Every demo should save a machine-readable result struct containing config, random seed, theoretical truth, estimate, bias/error, and software version/hash. Plots should be regenerable from saved results. This is especially useful when the simulation becomes part of a paper and we need reproducibility months later.

25 Recommended first coding sprint

1. Write config + unit conversion helper.
2. Implement analytic chirp phase and delayed phase.
3. Implement ideal dechirp.
4. Implement phase-slope estimator and FFT plotter.
5. Unit-test $f_b = S\tau$ with 5 ns, 100 ps, and 10 ps cases.
6. Implement reciprocal two-way wrapper and solve τ, θ .
7. Create three clean figures.
8. Only then decide whether to add CZT before showing Saeed.

This sprint is intentionally small enough to complete quickly while still producing a model we will keep.

26 Definition of done for the first Saeed deliverable

The first MATLAB deliverable is done when:

- equations in the code comments match the mathematical handbook;
- all ideal unit tests pass;
- fractional delay is not sample-quantized;
- a clean beat-frequency plot matches theory;
- two-way recovery returns injected TOF and clock offset;
- the plot captions explicitly state the ideal assumptions;
- the script runs from a clean MATLAB path with no manual variable editing.

Anything beyond this is optional for the first program-manager presentation.

27 What comes immediately after the first plots

The next scientifically useful task is not phase coding. It is to add independent frequency offset and up/down chirps, because that tests whether the simulator can distinguish epoch offset from oscillator mismatch in the same way the foundational cooperative work does. After that, compare beat estimators under AWGN. These two steps establish the backbone needed before phase noise and hardware calibration.

28 Final validation against hardware

When the second AWR2944 arrives:

1. characterize each board separately with the current package;
2. establish a shared or measured frequency reference strategy;
3. record direct line-of-sight cross-link data if hardware mode permits;
4. extract actual chirp/ADC parameters into the simulation config;
5. compare measured beat frequency/phase statistics to ideal simulation;
6. fit only one impairment class at a time (CFO/skew, PN, group delay, nonlinearity);
7. freeze a calibrated model before attempting coded operation.

The simulator should become a diagnostic digital twin, not merely a figure generator.

29 Reference anchors

The architecture follows the modular ideas in Scheiblhofer et al. 2008; the ideal time/frequency synchronization progression follows Roehr/Vossiek/Gulden 2007; double-sided processing is benchmarked against Gottinger et al. 2019; estimator design is anchored by Scherr et al. 2015; nonlinearity/phase-noise layers follow Ayhan 2016, Herzel 2018, and Tschapek 2022; coded extensions follow Lampel/Uysal/Kumbul; AWR parameter mapping follows current TI documentation in the archive.

30 Module contracts

Each module should have a small, documented input/output contract. Suggested contracts follow.

fmcw_phase(t, cfg)

Input: physical/local time vector and waveform config. **Output:** deterministic phase in radians. **Invariant:** derivative divided by 2π equals configured instantaneous frequency. **No noise inside this function.**

apply_fractional_delay(t, phase, tau)

Prefer evaluation of the analytic phase at $t - \tau$ for V0. Later, for arbitrary sampled waveforms, use a validated fractional-delay filter. **Invariant:** a sinusoid receives phase shift $-2\pi f\tau$ without amplitude distortion over the tested band.

dechirp(rx, lo)

Return complex product with an explicitly documented conjugation order. **Invariant:** one ideal delayed up-chirp produces a positive or negative pure tone exactly matching the selected convention.

estimate_phase_slope(z, Fs)

Return $\hat{f}$, phase intercept, fit residual, and a quality flag. Reject/flag records with excessive phase residual rather than silently returning a number.

solve_twtt(fAB, fBA, S)

Return $\hat{\tau}$, $\hat{\theta}$ plus the sign convention. This function should contain almost no DSP; it is algebra and therefore easy to test exhaustively.

31 Detailed unit-test matrix

ID	Test	Injected condition	Expected result
T01	zero delay	$\tau = 0$	zero timing beat (or known offset only)
T02	5-ns delay	ideal	$f_b = S\tau$
T03	100-ps delay	ideal	sub-bin estimator exact in noise-free case
T04	10-ps delay	ideal	no sample-quantization error
T05	half-sample delay	choose $\tau = 0.5/F_s$	exact analytic delay still recovered
T06	double slope	$S \rightarrow 2S$	f_b doubles
T07	negative slope	$S < 0$	beat sign changes as derived
T08	conjugate order	swap mixer convention	sign flips only

T09	amplitude scaling	$\alpha = 0.01, 1, 10$ noise-free	frequency unchanged
T10	initial phase	random ϕ_0	frequency unchanged
T11	two-way no offset	$\theta = 0$	$f_{AB} = f_{BA}$
T12	two-way clock offset	$\theta = 100$ ps	exact $\hat{\tau}, \hat{\theta}$
T13	negative clock offset	$\theta < 0$	sign recovered
T14	zero distance, nonzero offset	$\tau = 0$	two-way algebra isolates θ
T15	CFO up-chirp	$\Delta f \neq 0$	beat bias equals CFO
T16	CFO up/down	two slopes	recover $\Delta f, \tau$
T17	slope mismatch	$\Delta S \neq 0$	instantaneous beat has linear drift
T18	clock skew	$\epsilon \neq 0$	beat drift matches first-order model
T19	AWGN zero	SNR = ∞	identical to ideal baseline
T20	AWGN scaling	SNR sweep	variance decreases with SNR
T21	common PN	same PN TX/LO	range-correlation cancellation visible
T22	independent PN	independent oscillators	larger residual than common case
T23	systematic curvature zero	coefficient = 0	exact baseline
T24	sinusoidal ramp ripple	known ripple	expected beat sidebands/residual
T25	group-delay symmetry	equal paths	two-way bias common-mode
T26	group-delay asymmetry	unequal TX/RX	predicted clock-offset bias
T27	one multipath echo	known $\alpha, \Delta\tau$	multi-tone/vector effect visible
T28	ADC quantization disabled	infinite bits	baseline preserved
T29	clipping	low full scale	estimator quality flag trips
T30	code identity	two orthogonal codes	correct station selected
T31	code delay	fractional chip shift	naive correlation degrades
T32	code alignment compensation	same T31	recovery improves

32 First demo pseudocode

```
% demo01_single_link_ideal.m
cfg = cfg_awr_smoke_v1();
cfg.link.tau = 5e-9;
cfg.impair = disable_all_impairments(cfg.impair);

t = (0:cfg.N-1).' / cfg.Fs;
phi_tx = fmcw_phase(t, cfg.waveform);
phi_rx = fmcw_phase(t-cfg.link.tau, cfg.waveform);
tx = exp(1j*phi_tx);
rx = exp(1j*phi_rx);
beat = dechirp(tx, rx, cfg.convention);

fb_theory = cfg.S * cfg.link.tau;
res_phase = estimate_phase_slope(beat, cfg.Fs);
res_fft = estimate_fft_peak(beat, cfg.Fs);
```

```
assert(abs(res_phase.f - fb_theory) < cfg.test.freq_tol);  
plot_chirp_geometry(...);  
plot_beat_spectrum(...);  
  
disp(result_table(fb_theory,res_phase,res_fft));
```

The exact function names can change, but the separation between theory, waveform, estimator, assertion, and plotting should remain.

33 Two-way demo pseudocode

```
% demo02_twtt_ideal.m  
cfg.link.tau = 5e-9;  
cfg.clock.theta = 100e-12;  
  
dAB = cfg.link.tau + cfg.clock.theta;  
dBA = cfg.link.tau - cfg.clock.theta;  
  
zAB = simulate_ideal_link(cfg, dAB);  
zBA = simulate_ideal_link(cfg, dBA);  
  
fAB = estimate_phase_slope(zAB, cfg.Fs).f;  
fBA = estimate_phase_slope(zBA, cfg.Fs).f;  
  
[tau_hat, theta_hat] = solve_twtt(fAB, fBA, cfg.S);  
assert_close(tau_hat, cfg.link.tau);  
assert_close(theta_hat, cfg.clock.theta);
```

34 Estimator benchmark protocol

Use identical synthetic beat records for every estimator. For each SNR/beat frequency:

1. generate a clean complex tone or dechirped waveform;
2. add one frozen random-noise realization per trial and feed the same record to all estimators;
3. log frequency bias, frequency standard deviation, timing-equivalent error, phase error, failures, and runtime;
4. compare against a white-noise CRLB using the exact SNR convention;
5. repeat for off-bin frequencies uniformly distributed within one FFT bin to avoid favoring a bin-centered case.

Candidate estimators:

- nearest FFT bin (baseline only);
- parabolic/interpolated FFT peak;
- phase-slope least squares;
- CZT refinement;
- direct nonlinear complex-sinusoid fit;
- multi-chirp joint fit later.

35 Monte Carlo experiment matrix

A compact initial matrix is:

Sweep	Values	Output
SNR	-10:5:40 dB	bias/std/RMSE for τ, θ
clock offset	0, 10, 100 ps, 1 ns	linearity and sign
slope	10, 30, 60 MHz/us	timing sensitivity vs IF frequency
observation N	128,256,512,1024	estimator variance vs coherent time
CFO	0, 1, 10, 100 kHz	bias for single slope; recovery with up/down
slope mismatch	0, 1, 10, 100 ppm	tone-to-chirp transition and bias
PN level	nominal mask +/- 5/10 dB	timing precision and residual PSD
group asymmetry	0, 10, 100 ps	absolute TWTT bias
multipath	-10 to -40 dB, variable excess delay	phase/frequency bias

Do not run the entire cross-product at first; use one-dimensional sweeps with all other impairments disabled, then selected interactions.

36 AWR2944 config extraction checklist

When importing the current radar package profile, store both raw config values and derived SI values:

- raw start-frequency register/config value and derived Hz;
- raw slope value and derived Hz/s;
- ADC sample rate in ksps and Hz;
- ADC start time, ramp end time, idle time;
- samples/chirp, chirps/frame, frames, frame period;
- RX/TX masks and TX sequence;
- real/complex ADC format and I/Q convention;
- HPF corner selections;
- calibration/monitor settings that may affect repeatability;
- reference clock source and frequency;
- any trigger/sync mode used.

Write these into the saved result metadata so a plot always states exactly which hardware profile it represents.

37 DCA1000/raw-data considerations

The DCA1000 documentation reminds us that raw ADC data is captured over UDP and then reordered/zero-filled by tooling. For a precision timing experiment, packet loss cannot be treated as benign if zeros land inside the chirp used for phase estimation. The hardware pipeline should therefore record packet-loss/reorder statistics and reject or flag corrupted chirps rather than silently averaging them.

The parser must also preserve the exact I/Q ordering and RX-channel mapping. A swapped I/Q convention conjugates/sign-flips phase and can masquerade as a timing-sign bug.

38 Low-frequency IF feasibility checklist

Before choosing a 10-ps-centered beat near DC on hardware, verify:

1. configured analog HPF corner frequency;
2. digital DC/static-clutter removal in any firmware path;
3. RX DC offset/leakage magnitude;
4. usable IF passband and ADC window;
5. whether a known intentional frequency offset can move the timing beat to a cleaner band;
6. stability/knowledge of that offset across the two units.

A simulation option f_{off} should exist from the beginning even if set to zero in V0.

39 Two-board hardware characterization plan

Before attempting time transfer, run a characterization ladder:

H0 - independent monostatic sanity

Each board separately measures the same static corner reflector. Record range spectrum, beat phase across chirps, and temperature/power-cycle repeatability.

H1 - reference-clock experiment

If feasible from TI documentation/hardware, drive both with a common external reference. Measure whether chirp beat/frequency statistics improve compared with independent references. Do not assume carrier phase is coherent.

H2 - trigger alignment experiment

Use the available hardware/software trigger mechanism and measure chirp-start repeatability. If there is no direct time marker, infer relative start from a strong direct cross-link or external instrument.

H3 - one-direction cross-link

Transmit from A and receive/dechirp at B if the radar mode permits. Measure CFO/slope mismatch and compare with V2 simulation.

H4 - reciprocal exchange

Acquire both directions under the most simultaneous/controlled scheme available. Save raw complex data from both. Apply the ideal two-way solve and quantify bias/std.

H5 - calibration/repeatability

Reverse cables/positions where meaningful, power cycle, vary temperature/time, and determine which fixed delays are stable enough to calibrate.

40 Result-file schema

Save a MATLAB struct roughly like:

```
result.meta.timestamp
result.meta.git_commit
result.meta.matlab_version
result.cfg.raw_hardware
result.cfg.derived_si
result.truth.tau
result.truth.theta
result.signal.beat_AB
result.signal.beat_BA
result.estimator.name
result.estimate.fAB
result.estimate.fBA
result.estimate.tau
result.estimate.theta
result.quality.snr_db
result.quality.phase_fit_rms
result.quality.packet_loss
result.random.seed
```

For large Monte Carlo data, save aggregate statistics plus seeds/config rather than every waveform unless needed.

41 Plot style for Saeed/program manager

The first figures should answer one question each and have the theory printed on the figure or caption. Recommended visual hierarchy:

1. **Concept plot:** chirp slope + delay $\rightarrow$ frequency separation.
2. **Validation plot:** measured/simulated beat peak versus theoretical $S\tau$.
3. **TWTT plot:** two beats $\rightarrow$ recovered TOF and clock offset.

Avoid range-Doppler heatmaps or MIMO angle plots unless they directly support the timing story. Those are familiar radar visuals but distract from the novelty direction.

42 Software-quality gates

- Every demo script starts from a clean workspace and deterministic config.
- Every impairment has an enable flag and zero-effect test.
- No magic unit conversions inside equations.
- All random generators use recorded seeds.
- Plotting functions never alter data/estimates.
- All estimators return quality metrics, not only a scalar.
- A regression test stores several known configurations and expected estimates.
- Functions avoid dependence on unavailable toolboxes unless explicitly documented.

43 Milestone schedule from here

M0 - corpus complete.

Done with this archive: source map, equations, blueprint, deduped PDFs, and missing-source status.

M1 - ideal single-link.

MATLAB V0 + unit tests + two clean plots.

M2 - ideal TWTT.

reciprocal V1 + recovered τ, θ plot for Saeed.

M3 - independent clocks.

CFO/up-down + skew/slope mismatch.

M4 - precision baseline.

AWGN estimator benchmark and CRLB comparison.

M5 - physical realism.

phase noise, nonlinearity, IF/group delay, ADC.

M6 - AWR mapping.

config-driven simulation matched to actual capture settings.

M7 - two-board data.

calibrate/fit model against hardware.

M8 - coded extension.

node identity / phase-coded FMCW only after M7 backbone is stable.

44 Stop conditions and debugging order

If an estimate is wrong, debug in this order:

1. units (μs versus s, MHz/us versus Hz/s);
2. mixer sign/conjugation;
3. delay convention (one-way versus round trip);
4. sample/time vector origin and ADC start time;
5. estimator phase unwrap/fit;
6. CFO/slope terms;
7. only then stochastic noise/nonlinearity.

Most spectacular FMCW simulation errors are bookkeeping errors before they are radar-physics errors.